

Is a Diagnosis Worth Verifying? Measuring the Cost, Noise and Payoff of Patch Verification in Self-Improving Agents

Anonymous ACL submission

Abstract

Self-improving LLM agents let an analyst model read failed trajectories, blame a component and patch it, and recent systems verify each patch before applying it. We measure what this verification costs and what it buys, for one agent model and one analyst model on three customer-service tool-use settings: a fault-injected environment we built and the retail and airline domains of τ^2 -bench. Four findings organise the paper. First, none of the evidence we tested was cheap. A single-step check that regenerates the failing step and asks an LLM judge costs 22–72% of a full replay and detects 32–71% of real repairs; shorter judge prompts and a smaller judge model lower the judge’s share, but regenerating the step alone costs 12–15% of a replay, and none of four judge configurations is more informative per unit cost than a replay. Second, replays need controls: 20–23% of unpatched replays from the failing step succeed anyway. Third, a patch’s effect on previously solved tasks decides which verification policy is best, and it is the least measurable quantity: a task-matched unpatched baseline places its mean at -0.003 (retail) and -0.006 (airline), with 90% intervals of about ± 0.03 , but per-patch effects are not identified, and no policy is reliably best on our intervention banks, whether policies and outcomes use the same bank or separate halves of it. Fourth, on held-out tasks, applying the attributed patches without verification raised success from 0.76 to 0.88 on retail and from 0.68 to 0.75 on airline, and no verified patch set did better in this small-budget experiment; under the measured uncertainty, the probability that ten episodes of verification ever pay back is at most 0.56. To organise these measurements we decompose what verification can be worth into avoiding harm, choosing among candidate patches and discovering missed fixes. The decomposition is an accounting framework whose values depend on the harm model, not a decision rule. We release the environments, fault suites and

intervention banks.

1 Introduction

A growing family of systems improves LLM agents from their own failures: the agent runs, an analyst model reads the failed trajectories, and its diagnosis drives a change to a prompt, a tool description or the orchestration (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026; Chen et al., 2026). The diagnosis step, *failure attribution*, is unreliable (Zhang et al., 2025b; Liu et al., 2026): models label failures by their visible symptom rather than their cause, so a loop that trusts them will often change the wrong component. The common remedy is to verify before acting: patch the blamed component and replay the failure (Ma et al., 2025; Lin et al., 2026; Shah, 2026), ask a judge whether the failing step is fixed, or re-run tasks that used to pass and keep the patch only if nothing breaks (Chen et al., 2026).

Verification costs agent episodes. A tempting design, and the one we started from, treats LLM judgements as cheap evidence and full replays as expensive and allocates a budget across both; in a simulator built on these assumptions it needs half the budget of the strongest baseline (Appendix L). On real agents the premise failed for every judge we tried. Across three tool-use settings, a judge check that reads the trajectory costs 23–72% of a full replay and detects only 37–57% of real repairs; giving the judge only the failing step, or using the smaller agent model as judge, makes it cheaper but not more informative per unit cost, because regenerating the step is itself a sizeable part of a replay. None of the evidence we tested was cheap.

We therefore price every piece of evidence in episodes and ask a prior question: *when is a diagnosis worth verifying at all?* Our answer is empirical and specific to the agent, analyst and environments we study. Our contributions are:

- **Measurements of verification evidence** on ShopDesk and on offline intervention banks for τ^2 -bench retail and airline: paired targeted replays and judge checks for every patch, four judge configurations, null replays that expose spurious recovery, regression runs, and a task-matched unpatched baseline (§7).
- **An evaluation of eleven verification policies** in three ways: on the full banks, with policy evidence and outcome assessment taken from separate halves of each bank, and by deploying their patch sets on held-out tasks. A patch’s effect on previously solved tasks decides the ranking; its mean is within ± 0.03 of zero, but per-patch effects leave the ranking open, and on held-out tasks no gain from verification over applying the attributed patches was detected at budget 10 (§7).
- **An accounting framework.** A patch’s net effect splits into repair, spurious recovery and regressions, each estimated by one kind of experiment (§3), and what verification can add splits into avoiding harm, choosing among candidate patches and discovering missed edges (§4). We state which guarantees survive deployment constraints. The framework says where verification could pay; it is not an operational decision rule, because its decisive input, the harm of each individual patch, is what our experiments could not measure, and its values change several-fold with the harm model.

A net-effect allocation method built from these measurements serves as a reference policy (§5); it is not best everywhere, and CARVE, the multi-fidelity design we started from, is reported as a design study in Appendix L.

2 Related Work

Failure attribution for LLM agents. Who&When and its extension show that locating the responsible agent and step remains far from solved (Zhang et al., 2025b; Liu et al., 2026). AgenTracer trains a small attribution model on counterfactual replays and injected faults (Zhang et al., 2026); AgentDebug and ErrorProbe pair taxonomies or dependency tracing with corrective feedback (Zhu et al., 2025; Li et al., 2026). MAST gives attribution a fixed vocabulary (Cemri et al., 2025), and AdaMAST induces a system-specific

taxonomy validated by agreement between LLM annotators (Cemri et al., 2026); we instead check the mapping from categories to components by intervention.

Verification by intervention. DoVer validates hypotheses by editing a message or plan and re-running (Ma et al., 2025); REFLECT tests patches by targeted replay (Lin et al., 2026); CausalFlow and Causal Agent Replay score or formalise step interventions causally (Bonagiri et al., 2026; Shah, 2026); AgentDebugX gates reruns by policy (Zhu et al., 2026). These operate per trajectory or hypothesis and do not measure what the evidence costs or allocate a shared budget, which is our focus; any of them can serve as the replay primitive.

Self-improving agents. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); ProTeGi, OPRO, TextGrad and DSPy rewrite prompts from feedback (Pryzant et al., 2023; Yang et al., 2024; Yuksekogonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over designs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026). HarnessFix promotes a repaired harness only if it solves more validation tasks while breaking at most two (Chen et al., 2026), a regression-aware gate on whole bundles; Bayesian-Agent applies threshold rules to Beta posteriors over skills (Wu et al., 2026). We compare against HarnessFix-style gates and ask whether any of this verification pays.

Experimental design. Our reference policy uses the knowledge gradient (Frazier et al., 2008; Frazier and Powell, 2010) within Bayesian experimental design (Chaloner and Verdinelli, 1995); related problems include multi-fidelity bandits (Kandasamy et al., 2016; Poloczec et al., 2017), best-arm identification (Locatelli et al., 2016; Audibert et al., 2010; Russo, 2016), sequential testing (Wald, 1945) and causal bandits (Tong and Koller, 2001; Hauser and Bühlmann, 2014; Lattimore et al., 2016). Delta debugging isolates failure-inducing changes by experiment (Zeller and Hildebrandt, 2002), and program repair warns that a patch passing a failing test may not fix the fault (Smith et al., 2015).

3 Problem: Episode-Priced Evidence

Setting. An agent is assembled from *components* j (prompt sections, tool descriptions, configura-

tion). On a task distribution it fails with rate F ; an LLM groups its failures into categories i with shares f_i . For each *cell* (i, j) that the analyst blames, it proposes K candidate patches k that rewrite component j . Let $e_{ij} \in \{0, 1\}$ indicate whether changing j can repair category i at all, and q_k the fraction of category- i failures that patch k repairs if so. Separating e from q matters because a patch that fails to help may be aimed at the wrong component or merely be a bad patch. We call a cell with $e_{ij} = 1$ an *effective edge*. This is an interventional notion: some change to j repairs failures of category i . It need not mean that j was the original cause of those failures, and our ground truth (§6) establishes only that a tested patch helps.

Net effect. Applying patch k of cell (i, j) changes the success rate by

$$\Delta_k = F f_i e_{ij} q_k - (1 - F) \rho_k - c_a - c_w (1 - e_{ij}), \quad (1)$$

where ρ_k is the patch’s absolute excess failure rate on tasks that used to succeed (regressions, which may hit any category): such a task fails a re-run with probability r_0 unpatched and $r_0 + \rho_k$ patched, so $\rho_k < 0$ is possible, $c_a \geq 0$ is a cost charged for every applied change (review, maintenance) and $c_w \geq 0$ a penalty charged only for a change aimed at a wrong component (risk not captured by ρ_k). We report costs as fractions of the failure mass F and vary both. The goal is to choose a set of patches, at most one per cell, that maximises $\sum_k \Delta_k$ on held-out tasks.¹

Evidence. Three kinds of experiment, each costing a fraction of an episode, inform Eq. 1:

- A *targeted replay* resumes a category- i failure from the attributed step with the patch applied; it succeeds with probability $b_i + (1 - b_i) e_{ij} q_k$.
- A *null replay* does the same without a patch and succeeds with probability b_i , the category’s *spurious recovery* rate. Without it, replay success cannot be interpreted.
- A *regression run* executes a previously successful task from scratch with the patch and fails with probability $r_0 + \rho_k$, where r_0 is the natural re-run failure rate.

A fourth, a *single-step check* that regenerates only the failing step and asks an LLM judge whether

¹Effects of patches to different components are treated as additive. Held-out success of whole patch sets and a leave-one-out ablation (§7, Appendix I) are the only checks.

the error is gone, is often proposed as the cheap option. We measure its cost and accuracy in §7. Unlike classic multi-fidelity settings, all four cost a substantial fraction of an episode, within a factor of four of each other.

4 When Can Verification Pay?

Before any evidence, the net effects $\Delta = (\Delta_k)$ of the candidate patches are uncertain, with a prior that reflects the analyst’s accuracy and what is known about patch quality and regressions. The *default* applies, in each cell, the patch with the largest $\mathbb{E}[\Delta_k]$ if that is positive. Verification observes outcomes Y and then chooses; its value is $V(Y) = \mathbb{E}[\max_S \mathbb{E}[\sum_{k \in S} \Delta_k \mid Y]] - \max_S \mathbb{E}[\sum_{k \in S} \Delta_k]$, before the cost of the experiments. Write $x^+ = \max(x, 0)$ and $x^- = \max(-x, 0)$.

Proposition 1 (Value of verification). *For any experiments Y , $V(Y)$ is at most the value of learning Δ exactly, which is the sum over cells of*

$$V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \max_k (\mathbb{E} \Delta_k)^+. \quad (2)$$

With k^* the default patch of the cell,

$$V_{ij} \leq \underbrace{\mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+]}_{\text{choosing among patches}} + \underbrace{\mathbb{E}[\Delta_{k^*}^-]}_{\text{avoiding harm}}, \quad (3)$$

with equality when the default applies k^* , and $\mathbb{E}[\Delta_{k^*}^-] \leq (1 - F) \mathbb{E}[\rho_{k^*}^+] + c_a + c_w P(e_{ij}=0)$.

Corollary 1 (Edge-only verification). *Suppose a cell has a single candidate patch whose net effect depends only on e_{ij} : $\Delta = G$ if $e_{ij} = 1$ and $\Delta = -H$ if $e_{ij} = 0$, with $G, H \geq 0$ known (so q_k and ρ_k are known given the edge). With $\pi = P(e_{ij}=1)$, $V_{ij} = \min(\pi G, (1 - \pi)H)$.*

Proofs are in Appendix A. *What survives deployment constraints.* Deployment may use a heuristic default D , applies at most one patch per component, which couples cells, combines repairs of one category as $1 - \prod_k (1 - q_k)$ (overstated by at most $f_i \sum_{k < l} q_k q_l$ when added), and cannot push the failure rate on solved tasks below zero ($-\sum_k \rho_k \leq r_0$). Three statements survive. (i) The first sentence of the proposition holds for any set of admissible actions, so $V(Y)$ is at most the value of perfect information over deployable sets. (ii) Over a heuristic default that value is $V + G_D$, where $G_D = \max_S \mathbb{E}[\sum_{k \in S} \Delta_k] - \mathbb{E}[\sum_{k \in D} \Delta_k] \geq 0$ is the *decision-rule gap* that better use of the prior alone would close. (iii) The value of dropping

harmful default patches is at most their expected harm, $\sum_{k \in D} \mathbb{E}[(1 - F)\rho_k^+ + c_a + c_w(1 - e_{ij})]$, because dropping a patch from an overlapping set loses at most its marginal repair. What does not survive is the separation over cells: with one patch per component, the choice and discovery terms have no closed form, and we compute them in sequence (harm, then choice on the default’s components, then any component) with a greedy optimiser, which gives a lower bound on the constrained value of perfect information under a given harm model (§7). The three sources of value are *avoiding harm*, which needs regressions or change costs because replay success alone cannot create it; *choosing among patches* of different quality; and *discovery* of edges the analyst missed, the whole term $\mathbb{E}[\max_k \Delta_k^+]$ in cells the default leaves alone. The corollary fails when quality or regressions are uncertain given the edge ($\pi = 1$ with $\Delta = +0.10$ or -0.05 equally likely gives $V = 0.025$).

Proposition 2 (Cost of evidence). *Under Wald’s approximations, a sequential test that decides e_{ij} from targeted replays with error δ needs on average about $\log \frac{1-\delta}{\delta} / \text{KL}(b + (1-b)q \parallel b)$ replays when the edge is real. Detecting a regression of size ρ against a base rate r_0 with a one-sided level- α test and power $1 - \beta$ needs about $(z_\alpha + z_\beta)^2 r_0(1 - r_0) / \rho^2$ regression runs.*

Two consequences follow. First, targeted replays are uninformative about the harm term, because they only see the target category: a patch can pass every replay and still break other tasks. Second, small regressions are expensive to see. With $r_0 = 0.1$, a regression of $\rho = 0.25$ is detected with 90% power by 20 runs, whereas $\rho = 0.03$ needs about 850. Since many small regressions add up, failing to detect them is not evidence that they are absent; §7 therefore reports interval estimates rather than significance counts.

Three questions. Proposition 1 is the standard value-of-information argument (Chaloner and Verdinelli, 1995); what is new is its decomposition for patches and its evaluation on measured banks. *Allocation* asks which experiments to run within a budget (§5); *whether to verify at all* depends on the horizon: experiments costing C episodes pay back over N future episodes when $N V(Y) > C$ in success units.

5 Allocation Policies

When verification is warranted, the budget must still be allocated across cells, patches and experiment types. We compare eleven policies.

Net-effect allocation (reference). A spike-and-slab model per cell takes its edge prior from the attribution counts (Eq. 6, Appendix L), with grid posteriors for q_k and Beta posteriors for each b_i and each patch’s regression failure rate r_k . It estimates Eq. 1 as $P(e_{ij}=1 \mid \mathcal{D}) F f_i \mathbb{E}[q_k \mid \mathcal{D}] - (1 - F) \max(0, \mathbb{E}[r_k \mid \mathcal{D}] - r_0) / (1 - r_0) - c$, a stricter objective than Eq. 1: it charges c on every applied change ($c_a = c$, $c_w = 0$, whereas we score with c_w), never credits a patch for lowering failures on solved tasks, and scales harm by $1/(1 - r_0)$; Appendix G ablates the last two. It repeatedly runs the targeted replay, null replay or regression run with the largest n -step knowledge gradient per unit cost, $n \in \{1, 2, 4, 8\}$ (Frazier et al., 2008), and applies every patch with positive posterior net effect.

Baselines. *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* tests hypotheses in order of blame with three replays and accepts on two successes, as in do-then-verify methods (Ma et al., 2025). *HarnessFix-style, per patch* adds two regression runs that must break nothing; these counts are ours. *HarnessFix-style, bundle gate* is closer to the original, which validates a whole bundle of repairs on held-out tasks and promotes it if it solves at least one more task and breaks at most two (Chen et al., 2026). We apply the *Apply attributed* bundle, draw validation episodes from the bank, and allow three attempts. *Uncertainty sampling* replays the cell with the largest frequency-weighted posterior uncertainty, with or without three cheap checks per patch first. CARVE, the multi-fidelity design we started from, and its variants are described in Appendix L.

6 Environments and Intervention Banks

Environments. *ShopDesk* is a customer-service environment we built, with a seeded database, six deterministic tools, exactly checkable answers and 11 named agent components. τ^2 -*bench retail and airline* (Yao et al., 2024; Barres et al., 2025) are customer-service domains with an LLM user simulator and database-state rewards; unlike τ^2 ’s telecom domain, they are not dual-control, since only

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	full bank			evidence A, scored on B			full bank			evidence A, scored on B		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.77	0.77	0.77	0.29	0.29	0.29	0.73	0.73	0.73	0.67	0.67	0.67
Bayes default (Prop. 1)	0.12	0.12	0.12	–	–	–	0.81	0.81	0.81	–	–	–
Replay-each	0.80	0.71	0.85	0.45	0.48	0.51	0.62	0.58	0.60	0.51	0.44	0.46
HarnessFix-style, per patch	0.38	0.77	1.00	0.24	0.53	0.55	0.17	0.48	0.56	0.12	0.40	0.48
HarnessFix-style, bundle	0.40	0.66	0.82	0.22	0.41	0.46	0.42	0.66	0.70	0.38	0.58	0.59
Uncertainty sampling	0.78	1.05	0.97	0.33	0.63	0.65	0.67	0.71	0.74	0.59	0.60	0.62
Uncertainty + checks	1.01	1.07	1.08	0.67	0.88	0.93	0.64	0.59	0.63	0.57	0.53	0.55
CARVE, replay only	0.65	1.00	1.03	0.19	0.72	0.67	0.76	0.74	0.83	0.67	0.63	0.67
CARVE	1.00	0.85	0.74	0.84	0.89	0.92	0.66	0.42	0.29	0.61	0.44	0.39
CARVE, measured check	0.65	1.00	1.03	0.19	0.72	0.67	0.75	0.73	0.82	0.65	0.63	0.67
CARVE, robust check	0.85	1.22	1.16	0.34	0.82	0.75	0.72	0.72	0.82	0.64	0.63	0.66
Net-effect (ours)	0.70	0.90	1.08	0.37	0.58	0.74	0.63	0.70	0.77	0.55	0.55	0.55

Table 1: Posterior expected value of each policy’s patch set on the τ^2 intervention banks at $c = 0$, as a fraction of the repair-only oracle; bold is the best per column. *Full bank*: policies and scoring use the same bank. Each of 300 draws takes every patch’s excess failure rate over its task-matched unpatched baseline from the hierarchical posterior, the baseline from a resample of its runs, and edges and repair rates from a bootstrap of the replays, and averages over 40 resampling seeds. *Evidence A, scored on B*: policies ran on one random half of every list of outcomes and are scored, in the same way, only on the other half (5 halvings $\times$ 100 draws), so no outcome a policy saw is used to judge it. Sets are reduced to one patch per component, and harm respects the support constraints (§6). The Bayes default is Proposition 1’s no-evidence rule under the net-effect method’s priors. Values above 1 arise when patches are estimated to reduce failures on previously solved tasks. Intervals are in Table 7; the same table with $c_w = 0.02$ is Table 6, and with only significant regressions charged, Table 5.

the agent acts through tools. We split each domain policy into sections and add every tool description and the step limit as components, giving 24 components for retail and 21 for airline. The agent is DeepSeek-V4-Flash. Attribution, taxonomy induction (Wan et al., 2024), patch generation and judging use DeepSeek-V4-Pro.

Faults that look like defects. Each training episode runs clean or with one injected fault, written as a plausible authoring mistake rather than an obvious artefact. In retail: a policy section allowing gift-card refunds, a cancellation rule without the reason check, a product-tool description that confuses item and product ids, and a tight step limit. In airline: a wrong bag allowance, a 7-day instead of 24-hour cancellation window, a missing basic-economy restriction, proactive compensation, and a tight step limit. Clean episodes contribute natural failures: 22 of 137 on retail and 28 of 112 on airline.

Banks. For each τ^2 domain we take 36 cells, the attributed cells in order of blame plus random unattributed cells of populated categories, and generate three patches per cell. For every patch we record 20 paired targeted replays and single-step checks. We also record 20 null replays per category and 4 regression runs per patch, extended to 20 for

the 121 patches that any of the original policies accepted. Policies run offline by drawing recorded outcomes from the bank, with replacement (without, as a check), with costs in measured tokens and full replay = 1; resampling seeds measure policy variability given the bank, not new evidence. *Adjudication*: a cell is an effective edge if its best patch’s replay success exceeds the category’s null rate by at least 0.2 (one-sided Fisher $p < 0.05$), giving 4 effective edges on retail and 7 on airline. Choosing the best of three patches on the same replays makes this label optimistic, so we also score policies on bank halves they did not see. *Baseline*: we ran the unpatched agent 10 times on each of the 111 regression tasks (1,110 runs), about 12 hours after the regression runs; unpatched held-out success did not change over that time (0.816 vs. 0.817 on clean retail episodes). *Harm*: we score the posterior expected value of each patch set, drawing every patch’s ρ_k , its excess failure rate over its task-matched baseline, from a hierarchical normal posterior with estimated mean μ and spread τ , and edges and repair rates from a bootstrap of the replays. Draws respect the support: $\rho_k \geq -r_{0,k}$ and $\sum_k \rho_k \geq -\bar{r}_0$. Appendix G states and checks the model. *Deployable sets*: every accepted set is reduced to one patch per component, keeping the cell of the most frequent category, as on held-out

	ShopDesk	Retail	Airline
Analyst accuracy $\hat{r}$	0.72	0.41	0.29
Effective edges among blamed cells	–	0.18	0.33
Judge check: cost / replay	0.72	0.28	0.23
sensitivity	0.57	0.37	0.41
false-positive rate	0.21	0.11	0.13
break-even cost ratio	0.08–0.13	0.09	0.10
Spurious recovery b (mean)	–	0.23	0.20
range over categories	–	0–0.50	0–0.50
Repair q of effective edges (median)	–	0.50	0.41
Replays to decide an edge	–	5	7
Re-run failure r_0 , paired trials	–	0.11	0.14
unpatched, regression tasks	–	0.14	0.12
Patches harmful / helpful (95%)	–	2 / 5	3 / 4
$\rho > 0.10$ excluded / within ± 0.10	–	18 / 3	16 / 6
Pooled ρ , 90% interval	–	[−0.04, 0.04]	[−0.04, 0.02]
Mean μ , 90% interval	–	[−0.03, 0.02]	[−0.03, 0.02]
Spread τ of ρ_k	–	0.04	0.00

Table 2: Measured quantities. “Effective edges among blamed cells” is the precision of applying every attributed patch. The break-even cost ratio is the cost below which judge checks are more informative per unit cost than replays (Appendix L). Regression rows use the 121 patches with 20 regression runs against the task-matched baseline; exclusion is one-sided 95%, “within” is TOST at 0.05.

tasks; all primary results score these sets. Charging only significant regressions (one-sided binomial $p < 0.05$) is a sensitivity analysis. Values are fractions of the *repair-only oracle*, the failure mass repaired by the effective edges’ best patches with no cost, a fixed yardstick worth 0.070 (retail) and 0.114 (airline) in absolute success. Independently of the bank, we apply the accepted patch sets to 40 unseen retail tasks and 20 unseen airline tasks.

Protocol. Retail was the development domain for every method, including the variants in Appendix J. Airline was evaluated once for the original policies and once, later, for the net-effect method, which was frozen on retail beforehand.

What policies know before the budget. Attributions, taxonomy and candidate patches are shared and uncharged; every edge prior uses analyst accuracy $\hat{r} = 0.7$ rather than the audited values, and experiment costs, mean spurious recovery and judge calibration come uncharged from the bank. Regression tasks are drawn from solved clean training episodes, and 98–100% of the patches any policy deploys have 20 regression runs. Appendix B gives details.

7 Results

7.1 What evidence costs and what it shows

The evidence we tested is not cheap. The bank’s judge check reads the trajectory with the analyst

model. It costs 23–72% of a full replay and recognises only 37–57% of real repairs, so its information per unit cost is below that of a replay in every environment: the break-even cost ratio of Proposition 4 is about 0.1, and the measured ratio is 0.23 or more. Its false-positive rate is no higher for bad patches than for good ones: the judge is weak, not misled. Three cheaper judges, tested on 360 bank pairs per domain (Table 8, Appendix F), do not change this. Showing only the failing step cuts the judge’s share to 0.10–0.15 of a replay, but regenerating the step already costs 0.12–0.15; the agent model as judge detects more repairs (0.43–0.71) with more false positives (0.14–0.36). No configuration reaches its break-even ratio. A judge with false-positive rate 0.1 would need sensitivity above 0.55–0.63 at the cheapest prompt’s cost; none of ours came close. Caching, batching and deterministic checks were not tested.

Replays need controls. Without any patch, 20–23% of replays from the failing step succeed anyway, and the rate ranges from 0 to 0.5 across categories. Deciding one edge takes 5–7 replays even with the right b_i .

No average harm is resolved, and per-patch harm is not measured. Against the task-matched unpatched baseline, 5 of 121 patches are significantly harmful and 9 helpful on previously solved tasks (one-sided 95%), about what chance would produce. Pooled over all patches, the excess failure rate is -0.003 on retail and -0.006 on airline, with 90% intervals of ± 0.03 – 0.04 . With r_0 from paired training re-runs instead, the intervals were twice as wide, and on retail that r_0 was biased low (0.09 vs. 0.14). Only 3–6 patches per domain are shown to lie within ± 0.10 (TOST), and on retail the hierarchical model estimates a spread of $\tau \approx 0.04$ between patches, about 0.1 of the failure mass, comparable to what an effective edge repairs; measuring it per patch would take hundreds of runs (Proposition 2).

7.2 Where could verification add value?

Table 4 evaluates Proposition 1 on deployable sets. Every value in it is conditional on a model of the harm ρ_k that the regression runs cannot pin down per patch, so we report it under several models and read only its qualitative pattern. *Defaults.* The Bayes default, built from the net-effect method’s priors, applies 15–18 cells at $c = 0$ and does worse than the attributed-patches heuristic on re-

Patch set (budget 10)	Retail (40 tasks $\times$ 3)					Airline (20 tasks $\times$ 3)			
	All	Clean	No step fault	– Apply, all [90%]	#p	All	Clean	– Apply, all [90%]	#p
Unpatched	0.76	0.82	0.81		0	0.68	0.76		0
Apply attributed	0.88	0.87	0.87	–	7.0	0.75	0.76	–	5.0
Replay-each	0.83	0.87	0.84	–0.04 [–0.10, 0.02]	6.0				
HarnessFix-style, per patch	0.83	0.90	0.84	–0.04 [–0.13, 0.04]	0.8	0.71	0.77	–0.04 [–0.12, 0.04]	1.0
HarnessFix-style, bundle	0.85	0.87	0.86	–0.03 [–0.05, –0.00]	5.6	0.72	0.77	–0.03 [–0.09, 0.03]	3.0
Uncertainty sampling	0.86	0.86	0.86	–0.02 [–0.07, 0.04]	5.8				
Uncertainty + checks	0.85	0.82	0.86	–0.03 [–0.08, 0.03]	5.8				
CARVE, replay only	0.88	0.87	0.88	0.00 [–0.05, 0.06]	6.0				
CARVE	0.84	0.81	0.84	–0.04 [–0.10, 0.03]	5.2	0.74	0.72	–0.01 [–0.09, 0.07]	4.2
CARVE, measured check	0.85	0.86	0.85	–0.03 [–0.08, 0.02]	6.0				
CARVE, robust check	0.86	0.84	0.87	–0.01 [–0.06, 0.04]	6.0				
Net-effect (ours)	0.83	0.83	0.83	–0.04 [–0.10, 0.01]	3.4	0.75	0.76	–0.00 [–0.09, 0.08]	3.6

Table 3: Held-out success, mean over the patch sets of 5 seeds; #p is the mean number of patches. Retail: 40 unseen tasks $\times$ 3 episodes (38 clean, 82 fault-injected, 12 with the step-limit fault). Airline: 20 unseen tasks $\times$ 3 episodes (25 clean), five policies. “– Apply” is the paired difference from Apply attributed over all episodes, with a task-clustered 90% interval.

	Default rule	Value	Gap	Harm	Choice	Discov.	Total
Retail	Apply, $c = 0$	0.77	–0.64	0.87	0.98	0.17	2.02
	Bayes, $c = 0$	0.12	–	2.17	0.50	0.00	2.66
	Apply, $c_w = 0.02$	0.44	–0.17	1.05	0.87	0.02	1.95
	pooled ($\tau=0$)	1.12		0.23	0.22	0.42	0.87
	no pooling	–1.82		3.30	1.21	0.13	4.65
	tasks resampled	–0.52		2.28	0.90	0.13	3.31
	per cell	0.64		1.39	0.60	0.16	2.15
Airline	Apply, $c = 0$	0.73	0.08	0.04	0.25	0.40	0.69
	Bayes, $c = 0$	0.81	–	0.20	0.40	0.00	0.60
	Apply, $c_w = 0.02$	0.61	0.01	0.10	0.28	0.22	0.60
	pooled ($\tau=0$)	0.73		0.04	0.25	0.40	0.69
	no pooling	0.39		0.91	0.10	0.15	1.15
	tasks resampled	0.63		0.39	0.25	0.23	0.87
	per cell	0.86		0.15	0.11	0.30	0.56

Table 4: Value of perfect verification over two no-evidence defaults, deployable sets, posterior means as fractions of the repair-only oracle. *Gap*: Bayes default minus Apply attributed (G_D with the sign reversed; negative when the heuristic does better). Harm, choice and discovery are added in that order: dropping default patches with negative net effect, then the best cell and patch on the default’s components, then any other component. Indented rows give the value over Apply at $c = 0$ under other harm models (complete pooling, no pooling, tasks resampled before fitting; Appendix G) and with the additive per-cell form of Proposition 1 instead of deployable sets. Total of 1 is 0.070 (retail) or 0.114 (airline) in absolute success.

tail (0.12 vs. 0.77), about as well on airline (0.81 vs. 0.73): $G_D \geq 0$ holds only under the prior, so the negative realised gap means the edge prior is too generous. *Value*. Under the primary hierarchical model, perfect information is worth 2.0 oracles over the heuristic on retail (90% interval [0.3, 4.1], about 0.14 in absolute success) and 0.7 on airline ([0.3, 1.2], 0.08). Across harm models the retail value ranges from 0.9 (complete pooling) to 4.7 (no pooling), and airline’s from 0.7 to 1.2; dropping the support constraint, so that the uncertain

benefits of every unblamed patch add up without limit, gives 9.5 on retail. Only coarse features are stable: on retail, under every model but complete pooling, most of the value needs regression evidence (avoiding harm and choosing patches), and on airline the harm term stays at or below 0.4 except without pooling (0.9). Retail’s value is also sensitive to the patch harms the heuristic happens to apply: under no pooling or task resampling, the heuristic itself scores below zero.

7.3 Policies on the banks

Posterior expected value. Table 1 reports each policy’s posterior expected value for deployable sets. With μ near zero, the ranking is decided by which patches each policy happens to apply. On retail, CARVE with a robust check (1.22 at budget 40) and uncertainty sampling with checks (1.01 at 10) lead and applying everything scores 0.77; verifying policies beat it in 35–69% of draws, and with $c_w = 0.02$ applying everything falls to 0.44 (Table 6). On airline, applying everything (0.73) and CARVE with replays only (0.74–0.83) lead at $c = 0$, and applying everything is the single best policy in 18–40% of draws. Most intervals span about one oracle or more (Table 7), so the policies are statistically indistinguishable: what remains uncertain is each patch’s own ρ_k and the adjudication of edges.

Separating evidence from assessment. Reusing one bank for both the policies’ evidence and the outcome labels makes the labels optimistic for policies that select on the same replays. When policies see one half of every list of outcomes and are scored only on the other (right blocks of Table 1),

every value falls, because each half supports fewer effective edges (3–5 on retail, 3–9 on airline), but the picture does not change. On retail, applying everything scores 0.29 and the best verifying policies 0.84–0.93 (CARVE, uncertainty sampling with checks), yet they beat it in only 50–60% of draws, and most 90% intervals span two oracles or more. On airline, applying everything scores 0.67, no verifying policy exceeds it on average, and it is the single best policy in 30–39% of draws. Independent evidence on the banks therefore leaves the ranking as open as reused evidence does.

Robustness. Table 7 (Appendix E) rescores the budget-40 decisions. (i) *Adjudication is unstable.* On held-back halves only 1–3 of retail’s 4 effective edges and 3–7 of airline’s 7 are confirmed, and bootstrapping the replays yields anywhere from 2 to 19 (retail) and 4 to 19 (airline) edges. (ii) *Retail’s value is mostly the step limit.* Without the step-limit component, every retail policy falls below zero, while airline keeps 83% of its oracle. (iii) *Deployment constraints matter:* under significant-harm scoring, one patch per component raises applying everything from 0.16 to 0.47 (retail) and 0.15 to 0.46 (airline).

7.4 Held-out tasks

Table 3 applies each policy’s patch set to the agent on unseen tasks, which tests whole sets, interactions included, with runs independent of the bank. On retail, applying the attributed patches raises success from 0.76 to 0.88 ($+0.12 \pm 0.04$, paired, task-clustered); on airline, from 0.68 to 0.75 ($+0.07 \pm 0.06$). No improvement from verification was detected on either domain: every verified set differs from applying everything by -0.04 to 0.00 overall. This is not an equivalence result. With a non-inferiority margin of 0.05 in success, chosen after the fact, applying everything is non-inferior to 8 of the 10 verified sets on retail and to the two HarnessFix-style sets on airline, but not to CARVE or the net-effect method there, whose intervals reach $+0.08$. Retail ablations are in Appendix I: the step-limit patch helps only on step-limit-fault episodes, and on 158 clean episodes the sets are indistinguishable. This evidence is narrow (one budget, 40 and 20 tasks, five airline policies, mostly injected faults): it shows that no benefit was detected here, and its intervals still allow gains of up to $+0.06$ (retail) and $+0.08$ (airline).

From replays to deployment. Replays measure repair from the failing step; deployed patches act from the first step. The bank’s repaired failure mass times the held-out failure rate predicts airline’s held-out gains closely (Apply $+0.064$ vs. $+0.067$ observed, CARVE and net-effect within 0.001) but half of retail’s ($+0.054$ vs. $+0.117$), mostly because the step-limit patch adds $+0.07$ from the start against $+0.01$ predicted.

8 Discussion

The operating regime we can characterise. With analyst precision 0.18 – 0.33 , spurious recovery about 0.2 , a repairable gain of 0.07 – 0.11 , and patch effects on solved tasks of mean 0 ± 0.03 and spread ≤ 0.04 , perfect information about every patch is worth 0.06 – 0.33 (retail) and 0.08 – 0.13 (airline) in absolute success over applying the attributed patches, depending on the harm model. Whether verification captures any of it was tested only at budget 10. Drawing each verified set’s held-out gain from its task-clustered uncertainty, adding the change cost it saves by applying fewer patches ($c_a \sim U[0, 0.02]$), and charging its episodes once, the probability that it has paid back after 1,000 future episodes is 0.02 – 0.45 (retail) and 0.18 – 0.44 (airline), and that it ever pays back 0.05 – 0.56 and 0.23 – 0.51 (Table 11). In this regime ten episodes of verification are at best a coin flip. With a less precise analyst or larger harms, such as ShopDesk’s formatting patch (Appendix K), the harm term grows, and only regression runs and bundle tests measure it.

A framework, not a decision rule. Proposition 1 names the quantities that could make verification pay, but it cannot decide whether to verify a new diagnosis: its most influential input, each patch’s harm, takes hundreds of regression runs to measure (Proposition 2), and its values move several-fold with the harm model.

9 Conclusion

With our agent and the four judges we tried, no verification evidence was cheap, no average patch effect on solved tasks was resolved, policy rankings remain unidentified whether or not evidence and assessment share a bank, and on held-out tasks no gain from verification was detected at budget 10.

Limitations

We study one agent model and one analyst model on three customer-service settings, and four judge configurations built from those two models; caching, batching, fine-tuned or deterministic checks were not tested. The banks have 36 cells per domain, and their 4 and 7 effective edges are only partly confirmed on held-back replays; our edges are effective interventions, not established causes. Regression estimates rest on 20 runs per patch; the unpatched baseline (10 runs per task) was collected about 12 hours after them, and only held-out success checks for drift between the two; the hierarchical posterior assumes normally distributed patch effects and predicts more spread between patches than observed (Appendix G). Effects are treated as additive within a deployable set, apart from repairs in one category; whole sets are tested on held-out tasks at budget 10 only, with 60 episodes per set on airline, and the leave-one-out runs are too noisy to measure interactions. Most of retail’s held-out gain on fault-injected episodes comes from the step-limit fix. Most failures come from injected faults written to resemble real defects (natural failures are 16–25%), and reweighting to natural failures is not a bank built from them. We evaluate a single round of improvement under fixed budgets, not a multi-round loop or a deployment horizon. The net-effect method was developed on retail after we had seen airline results for the other policies, although not its own. All edge priors use an analyst accuracy of 0.7 rather than the audited values, and some inputs (experiment costs, mean spurious recovery, judge calibration) come uncharged from the bank. Replay from a failing step requires deterministic tools.

Ethics Statement

Self-improving agents change their own behaviour without a person reviewing each change. That no improvement from verification was detected on our held-out tasks, and that no average harm to other tasks was resolved, does not show that verification is unnecessary, and our measurements cannot exclude small harms per patch; it should not be read as licence to skip review. On ShopDesk a single patch silently broke a quarter of tasks (Appendix K). Our environments use synthetic customer data only.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.
- Jean-Yves Audibert, Sébastien Bubeck, and Rémi Munos. 2010. Best arm identification in multi-armed bandits. In *Proceedings of the 23rd Conference on Learning Theory*.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.
- Akash Bonagiri, Devang Borkar, Gerard Janno Andierias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.
- Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.
- Peter I. Frazier and Warren B. Powell. 2010. Paradoxes in learning and the marginal value of information. *Decision Analysis*, 7(4):378–403.
- Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.
- Alain Hauser and Peter Bühlmann. 2014. Two optimal strategies for active learning of causal models from interventional data. *International Journal of Approximate Reasoning*, 55(4):926–939.

744	Shengran Hu, Cong Lu, and Jeff Clune. 2025. Auto-	Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chen-	797
745	mated design of agentic systems. In <i>International</i>	guang Zhu, and Michael Zeng. 2023. Automatic	798
746	<i>Conference on Learning Representations</i> .	prompt optimization with “gradient descent” and	799
747	Kirthivasan Kandasamy, Gautam Dasarathy, Jeff	beam search. In <i>Proceedings of the 2023 Confer-</i>	800
748	Schneider, and Barnabás Póczos. 2016. The multi-	<i>ence on Empirical Methods in Natural Language</i>	801
749	fidelity multi-armed bandit. In <i>Advances in Neural</i>	<i>Processing</i> .	802
750	<i>Information Processing Systems</i> .	Daniel Russo. 2016. Simple Bayesian algorithms for	803
751	Omar Khattab, Arnav Singhvi, Paridhi Maheshwari,	best arm identification. In <i>Proceedings of the 29th</i>	804
752	Zhiyuan Zhang, Keshav Santhanam, Sri Vard-	<i>Conference on Learning Theory</i> .	805
753	hamanan, Saiful Haq, Ashutosh Sharma, Thomas T.	Jaineet Shah. 2026. Causal agent replay: Counterfactual	806
754	Joshi, Hanna Moazam, Heather Miller, Matei Zaharia,	attribution for LLM-agent failures. <i>arXiv preprint</i>	807
755	and Christopher Potts. 2024. DSPy: Compiling	<i>arXiv:2606.08275</i> .	808
756	declarative language model calls into self-improving	Noah Shinn, Federico Cassano, Ashwin Gopinath,	809
757	pipelines. In <i>International Conference on Learning</i>	Karthik Narasimhan, and Shunyu Yao. 2023. Re-	810
758	<i>Representations</i> .	flexion: Language agents with verbal reinforcement	811
759	Finnian Lattimore, Tor Lattimore, and Mark D. Reid.	learning. In <i>Advances in Neural Information Pro-</i>	812
760	2016. Causal bandits: Learning good interventions	<i>cessing Systems</i> .	813
761	via causal inference. In <i>Advances in Neural Informa-</i>	Edward K. Smith, Earl T. Barr, Claire Le Goues, and	814
762	<i>tion Processing Systems</i> .	Yuriy Brun. 2015. Is the cure worse than the disease?	815
763	Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-	overfitting in automated program repair. In <i>Proceed-</i>	816
764	Thu Le. 2026. Towards self-improving error di-	<i>ings of the 10th Joint Meeting on Foundations of</i>	817
765	agnosis in multi-agent systems. <i>arXiv preprint</i>	<i>Software Engineering</i> .	818
766	<i>arXiv:2604.17658</i> .	Simon Tong and Daphne Koller. 2001. Active learning	819
767	Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok,	for structure in Bayesian networks. In <i>Proceedings of</i>	820
768	Daniel Guo, Sahil Arun Nale, Charles Fleming,	<i>the 17th International Joint Conference on Artificial</i>	821
769	and Guang Cheng. 2026. REFLECT: Intervention-	<i>Intelligence</i> .	822
770	supported error attribution for silent failures in LLM	Rakshith Vasudev, Melisa Russak, Dan Bikel, and	823
771	agent traces. <i>arXiv preprint arXiv:2606.09071</i> .	Waseem Alshikh. 2026. Accurate failure prediction	824
772	Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tian-	in agents does not imply effective failure prevention.	825
773	wei Yue, Chi Wang, Jian Kang, Qingyun Wu, and	<i>arXiv preprint arXiv:2602.03338</i> .	826
774	Huazheng Wang. 2026. Who&when pro: Can LLMs	Abraham Wald. 1945. Sequential tests of statistical	827
775	really attribute failures in AI agents? <i>arXiv preprint</i>	hypotheses. <i>The Annals of Mathematical Statistics</i> ,	828
776	<i>arXiv:2607.09996</i> .	16(2):117–186.	829
777	Andrea Locatelli, Maurilio Gutzeit, and Alexandra Car-	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin	830
778	pentier. 2016. An optimal algorithm for the thresh-	Kim, Scott Counts, Jennifer Neville, Siddharth Suri,	831
779	olding bandit problem. In <i>Proceedings of the 33rd</i>	Chirag Shah, Ryen W. White, Longqi Yang, Reid	832
780	<i>International Conference on Machine Learning</i> .	Andersen, Georg Buscher, Dhruv Joshi, and Nagu	833
781	Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qing-	Rangan. 2024. TnT-LLM: Text mining at scale with	834
782	wei Lin, Saravan Rajmohan, and Dongmei Zhang.	large language models. In <i>Proceedings of the 30th</i>	835
783	2025. DoVer: Intervention-driven auto debug-	<i>ACM SIGKDD Conference on Knowledge Discovery</i>	836
784	ging for LLM multi-agent systems. <i>arXiv preprint</i>	<i>and Data Mining</i> .	837
785	<i>arXiv:2512.06749</i> .	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin,	838
786	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler	Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin	839
787	Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon,	Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent:	840
788	Nouha Dziri, Shrimai Prabhumoye, Yiming Yang,	Posterior-guided skill evolution across LLM agent	841
789	Shashank Gupta, Bodhisattwa Prasad Majumder,	harnesses. <i>arXiv preprint arXiv:2606.08348</i> .	842
790	Katherine Hermann, Sean Welleck, Amir Yazdan-	Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu,	843
791	bakhsh, and Peter Clark. 2023. Self-refine: Iterative	Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024.	844
792	refinement with self-feedback. In <i>Advances in Neu-</i>	Large language models as optimizers. In <i>Interna-</i>	845
793	<i>ral Information Processing Systems</i> .	<i>tional Conference on Learning Representations</i> .	846
794	Matthias Poloczek, Jialei Wang, and Peter I. Frazier.	Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik	847
795	2017. Multi-information source optimization. In	Narasimhan. 2024. τ -bench: A benchmark for tool-	848
796	<i>Advances in Neural Information Processing Systems</i> .	agent-user interaction in real-world domains. <i>arXiv</i>	849
		<i>preprint arXiv:2406.12045</i> .	850

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by back-propagating language model feedback. *Nature*, 639.

Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025b. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Proofs

Proposition 1. For any signal Y and any action set, $\max_S \mathbb{E}[U(S) \mid Y] \leq \mathbb{E}[\max_S U(S) \mid Y]$, so taking expectations gives $V(Y) \leq \mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]$, the value of perfect information; this is the standard argument and needs no assumption on how Y relates to Δ . With $U(S) = \sum_{k \in S} \Delta_k$ and at most one patch per cell, both maxima separate over cells: $\max_S U(S) = \sum_{ij} \max_k \Delta_k^+$ and $\max_S \mathbb{E}[U(S)] = \sum_{ij} \max_k (\mathbb{E}\Delta_k)^+$, which gives V_{ij} ; no independence between cells is needed. For Eq. 3, let k^* maximise $\mathbb{E}\Delta_k$. If $\mathbb{E}\Delta_{k^*} \geq 0$,

then $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \mathbb{E}[\Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^-]$, which is Eq. 3 with equality. Otherwise $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] = \mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^+]$, and $\mathbb{E}[\Delta_{k^*}^+] < \mathbb{E}[\Delta_{k^*}^-]$ because $\mathbb{E}\Delta_{k^*} < 0$. Finally, the repair term $F f_{ij} e_{ij} q_k$ is non-negative, so $\Delta_k^- \leq (1 - F)\rho_k^+ + c_a + c_w(1 - e_{ij})$; take expectations.

Heuristic defaults and constraints. For a default D , $\mathbb{E}[\max_S U(S)] - \mathbb{E}[U(D)] = (\mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]) + (\max_S \mathbb{E}[U(S)] - \mathbb{E}[U(D)])$, and the second bracket is $G_D \geq 0$ by definition of the maximum. Restricting S to deployable sets leaves the argument unchanged. For repairs in one category, the Bonferroni inequality gives $1 - \prod_k (1 - q_k) \geq \sum_k q_k - \sum_{k < l} q_k q_l$, which bounds the overstatement of the additive form. The bound on $-\sum_k \rho_k$ holds because the patched failure rate $r_0 + \sum_k \rho_k$ on solved tasks is non-negative under additivity. For (iii), removing patch k from a set S changes its value by $-\text{rep}(S) + \text{rep}(S \setminus k) + (1 - F)[\bar{\rho}(S) - \bar{\rho}(S \setminus k)] + c_a + c_w(1 - e_{ij})$, where rep is the product-form repair and $\bar{\rho}$ the capped sum of harms. Repair is monotone in S , so the first bracket is non-negative, and capping $\sum \rho$ from below changes it by at most ρ_k^+ when k is removed; summing over the dropped patches and taking expectations gives the bound. The greedy optimiser keeps, per component, the cell and patch with the largest positive net effect, which is a feasible deployable set, so the value it attains is a lower bound on the constrained optimum.

Corollary 1. With one patch, $V = \mathbb{E}[\Delta^+] - (\mathbb{E}\Delta)^+ = \pi G - (\pi G - (1 - \pi)H)^+ = \min(\pi G, (1 - \pi)H)$. The assumption that Δ is a function of e_{ij} is what makes the edge state sufficient: if q_k or ρ_k is uncertain given e_{ij} , learning only e_{ij} leaves value on the table and the general expression applies.

Proposition 2. The first part is Wald’s approximation for a sequential probability ratio test between $p_1 = b + (1 - b)q$ and $p_0 = b$: with thresholds $\pm \log \frac{1-\delta}{\delta}$, neglecting overshoot, the expected sample size under p_1 is the threshold divided by the per-observation drift $\text{KL}(p_1 \| p_0)$. The second is the normal-approximation sample size of a one-sided test of a binomial proportion, evaluated with the variance at r_0 .

B Policy Inputs and Regression Sampling

Attribution, taxonomy induction, labelling and candidate patches are shared by all policies and not charged to any budget. Every policy that uses an edge prior uses Eq. 6 with analyst accuracy $\hat{r} = 0.7$, not the audited 0.41 and 0.29 of Table 2; with the audited values the Bayes default is unchanged at $c = 0$, and at $c_w = 0.02$ it applies 2 cells instead of 4–6 and scores 0.33 (retail) and 0.30 (airline) of the oracle instead of 0.27 and 0.63. Three further inputs come from the bank itself and are also uncharged: the cost of each kind of experiment (mean tokens), the mean spurious recovery $\hat{b}$, which CARVE and uncertainty sampling plug in and the net-effect method uses only as a prior mean for its per-category b_i , and, for the two CARVE variants that use them, the judge’s measured sensitivity and false-positive rate. The re-run failure rate r_0 comes from paired re-runs of training tasks. *Regression tasks* are drawn uniformly, with replacement, from the clean training episodes the agent solved, so they follow the training-task distribution of the same domain, not the held-out tasks, which also include fault-injected episodes. The extension to 20 runs covered every patch an original policy accepted; 98–100% of the patches that any policy, including the later net-effect method, deploys at budget 40 have 20 runs, and a policy asking for more runs of a 4-run patch resamples those 4.

C Reproducibility

The code, environments and banks are submitted as anonymised supplementary material. It contains the environments and fault suites (agent_exp/env.py, agent_exp/tau2_env.py), every prompt used for attribution, taxonomy induction, labelling, patch generation and judging (agent_exp/analyst.py), the agent and analyst settings (DeepSeek-V4-Flash and DeepSeek-V4-Pro through the provider’s Anthropic-compatible endpoint, at most 4,000 output tokens per agent call and 8,000 per structured analyst call, provider-default sampling), and the complete intervention banks (runs/tau2_*/bank.json: every replay, judge check, null replay and regression run with its token count and source trace). Bank construction is deterministic given the seeds in agent_exp/bank.py: cells are the attributed cells in order of blame plus random unattributed cells of populated categories, and replay traces and regression tasks are drawn with fixed seeds. Every table

in the paper is produced offline from the banks by agent_exp.reanalysis, reanalysis3, reanalysis4, heldout_breakdown, review2_report and report_tau2; the judge configurations of Appendix F come from agent_exp.judge_configs; the new runs of Appendix I, the unpatched baseline and the airline held-out runs come from agent_exp.review2_runs and agent_exp.bank_heldout; the README lists the commands. Re-collecting the banks needs API access and is not bit-reproducible, because the provider does not offer seeded sampling.

D Significance-Thresholded Scoring

Charging only regressions significant against the paired-trial r_0 reproduces the picture of a scoring that sets uncertain harm to zero: applying everything recovers 84% and 60% of the oracle and no verifying policy reaches it. With $c_w = 0.02$ per wrong change, the per-patch HarnessFix-style rule (retail, budgets 10–40; airline, 40) and the net-effect method (retail, 80; airline, 80, tied with uncertainty sampling with checks) lead, and CARVE declines with budget on airline (0.30→0.19). Appendix H varies c for both cost types; the winner changes with c .

E Robustness at Budget 40

F Judge Configurations

The three new configurations regenerate the failing step once per pair and judge it with each prompt; the short prompt shows the task, the diagnosed root cause, the tool results or user message just before the failing step, and the old and new actions. 90% Wilson intervals on 180 positives and 180 negatives are about ± 0.06 for sensitivity and ± 0.03 –0.06 for the false-positive rate; on the 40-pair subsample they are about ± 0.16 . The short-prompt judges agree with the bank judge on 64–76% of pairs. The experiment used about 310 episode-equivalents of tokens. One pattern is consistent across both domains: the agent model is more permissive than the analyst model with either prompt, raising both sensitivity and false-positive rate, so its break-even ratio stays near 0.1 except for the full-trajectory prompt on airline (0.24, on 40 pairs), which costs 0.58 of a replay. Raising the judge’s sensitivity while keeping its false-positive rate low, rather than shortening its prompt, is what a cheap check would need.

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	significant harm			+ $c_w=0.02$			significant harm			+ $c_w=0.02$		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.84	0.84	0.84	0.16	0.16	0.16	0.60	0.60	0.60	0.15	0.15	0.15
Replay-each	0.78	0.74	0.71	0.14	0.33	0.36	0.43	0.09	0.10	0.07	-0.12	-0.08
HarnessFix-style, per patch	0.52	0.51	0.52	0.52	0.42	0.30	0.18	0.40	0.43	0.18	0.37	0.31
HarnessFix-style, bundle	0.11	0.21	0.26	-0.21	-0.31	-0.31	0.34	0.53	0.52	0.07	0.12	0.11
Uncertainty sampling	0.42	0.50	0.31	0.01	0.23	0.01	0.56	0.50	0.51	0.28	0.25	0.25
Uncertainty + checks	0.14	0.47	0.48	-0.25	0.25	0.24	0.55	0.50	0.53	0.27	0.29	0.36
CARVE, replay only	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.22	0.35	0.48	-0.16	-0.02	0.12
CARVE	0.33	0.32	0.52	-0.01	0.19	0.47	0.55	0.34	0.25	0.30	0.23	0.19
CARVE, measured check	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.25	0.43	0.54	-0.09	0.07	0.20
CARVE, robust check	0.34	-0.03	0.02	-0.15	-0.49	-0.48	0.22	0.34	0.48	-0.12	-0.01	0.14
Net-effect (ours)	0.24	0.52	0.66	0.09	0.41	0.52	0.26	0.39	0.46	0.14	0.31	0.36

Table 5: Sensitivity analysis: as Table 1, but accepted patches are charged only their significant measured regressions (one-sided binomial $p < 0.05$ against r_0), mean over 200 resampling seeds of the original bank. This scoring sets uncertain harm to zero and so favours policies that apply many patches.

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	full bank			evidence A, scored on B			full bank			evidence A, scored on B		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.44	0.44	0.44	-0.04	-0.04	-0.04	0.61	0.61	0.61	0.54	0.54	0.54
Bayes default (Prop. 1)	0.27	0.27	0.27	-	-	-	0.63	0.63	0.63	-	-	-
Replay-each	0.54	0.49	0.64	0.19	0.25	0.23	0.51	0.49	0.50	0.37	0.33	0.33
HarnessFix-style, per patch	0.38	0.70	0.82	0.24	0.46	0.38	0.16	0.44	0.51	0.10	0.36	0.40
HarnessFix-style, bundle	0.24	0.39	0.53	0.08	0.14	0.18	0.35	0.55	0.58	0.30	0.46	0.47
Uncertainty sampling	0.52	0.87	0.78	0.09	0.45	0.43	0.61	0.66	0.68	0.52	0.52	0.53
Uncertainty + checks	0.76	0.93	0.95	0.42	0.73	0.77	0.58	0.54	0.59	0.49	0.46	0.47
CARVE, replay only	0.35	0.74	0.70	-0.10	0.45	0.31	0.68	0.66	0.74	0.58	0.53	0.55
CARVE	0.79	0.75	0.70	0.62	0.76	0.81	0.59	0.36	0.25	0.53	0.34	0.30
CARVE, measured check	0.35	0.74	0.70	-0.10	0.45	0.31	0.67	0.64	0.74	0.56	0.53	0.55
CARVE, robust check	0.55	0.94	0.83	0.05	0.54	0.40	0.65	0.64	0.74	0.56	0.53	0.55
Net-effect (ours)	0.56	0.80	0.94	0.26	0.49	0.61	0.59	0.65	0.71	0.48	0.48	0.46

Table 6: As Table 1, with a cost of $c_w = 0.02$ of the failure mass charged per patch aimed at a wrong edge.

G Harm Model, Checks and Sensitivity

Model. For patch k with n_k regression runs and m_k failures, let $r_{0,k}$ be the unpatched failure rate of the tasks its runs used, averaged with the same task weights (10 unpatched runs per task). We set $y_k = m_k/n_k - r_{0,k}$ and use a normal approximation $y_k \sim N(\rho_k, v_k)$ with $v_k = \bar{p}(1 - \bar{p})/n_k$, $\bar{p}$ the pooled patch failure rate, and $\rho_k \sim N(\mu, \tau^2)$. τ^2 is estimated by the method of moments and truncated at zero; given τ^2 , μ has a flat prior and a normal posterior, and each ρ_k the conjugate normal posterior; patches without runs take $\rho_k = \mu$. Each of 300 draws resamples the unpatched runs within each task (a per-task Jeffreys posterior would add about 0.5/11 to every task without failures and bias the baseline upwards), draws μ and ρ , clips ρ_k at $-r_{0,k}$, and caps the set’s total at $-\bar{r}_0$. Tasks are shared across patches; the *tasks resampled* variant resamples tasks before fitting, which propagates

that dependence.

Posterior predictive check. On the patches with 20 runs, the observed standard deviation of failure rates is 0.076 (retail) and 0.066 (airline); replicated data from the fitted model give 0.092 and 0.081, with $P(\text{replicated} \geq \text{observed}) = 0.95$ and 0.97, and the same for new patches drawn from $N(\mu, \tau^2)$. The model therefore predicts more spread between patches than the data show, which makes harm uncertainty, and the harm and choice terms of Table 4, conservative (too large). The number of patches with no failure and the largest failure rate are reproduced (posterior predictive p of 0.91 and 0.99 on retail, 0.73 and 0.93 on airline, so the largest rates are slightly overpredicted as well).

H Change-Cost Sweep

Table 12 varies the change cost. The best policy changes with c : applying the attributed patches is

Policy (budget 40)		Posterior, $c_w=0.02$		Sensitivity: significant harm only						
		Value [90%]	$P(> \text{apply})$	c_w	c_a	Split	Natural	No step limit	One per comp.	No repl.
Retail	Apply attributed	0.44 [−1.68, 2.40]	–	0.16	0.01	−0.46	−1.25	−0.61	0.47	0.16
	HarnessFix-style, per patch	0.70 [0.04, 1.38]	0.57	0.42	0.34	0.12	−0.82	−0.27	0.42	0.54
	HarnessFix-style, bundle	0.39 [−1.11, 1.89]	0.44	−0.31	−0.42	−0.80	−1.99	−0.83	−0.08	−0.45
	Uncertainty sampling	0.87 [−0.50, 2.15]	0.69	0.23	0.09	−0.13	−1.10	−0.64	0.29	0.28
	CARVE	0.75 [−0.09, 1.65]	0.59	0.19	0.07	0.04	−0.89	−0.25	0.20	0.51
	Net-effect (ours)	0.80 [0.06, 1.52]	0.60	0.41	0.32	0.23	−0.86	−0.35	0.41	0.52
Airline	Apply attributed	0.61 [−0.13, 1.38]	–	0.15	−0.08	−0.41	−0.07	−0.03	0.46	0.15
	HarnessFix-style, per patch	0.44 [0.06, 0.86]	0.26	0.37	0.25	0.17	0.28	0.20	0.38	0.38
	HarnessFix-style, bundle	0.55 [−0.12, 1.25]	0.15	0.12	−0.09	−0.30	−0.09	−0.04	0.39	0.10
	Uncertainty sampling	0.66 [0.06, 1.32]	0.66	0.25	0.01	0.04	0.05	0.05	0.41	0.26
	CARVE	0.36 [−0.09, 0.83]	0.16	0.23	0.08	−0.03	0.24	0.22	0.25	0.23
	Net-effect (ours)	0.65 [0.06, 1.26]	0.58	0.31	0.14	0.17	0.23	0.13	0.35	0.26

Table 7: Uncertainty and robustness at budget 40 (fractions of the repair-only oracle). *Posterior*: deployable sets as in Table 1, mean and 90% interval over the hierarchical harm posterior and the replay bootstrap, and the probability of beating Apply attributed in the same draw. Sensitivity columns charge significant regressions and $c_w = 0.02$ per wrong change unless stated: c_a , 0.02 per applied change instead; *Split*, policies see a random half of every outcome list and adjudication uses the other half (5 splits); *Natural*, values reweighted to the category shares of natural failures; *No step limit*, the step-limit component removed from the bank; *One per comp.*, the accepted set reduced to one patch per component as deployed; *No repl.*, policies draw bank outcomes without replacement.

Judge	Retail					Airline				
	Cost	Sens.	FPR	Break-even	Ratio	Cost	Sens.	FPR	Break-even	Ratio
Bank judge (analyst, full trajectory)	0.29	0.37	0.07	0.14	0.47	0.22	0.43	0.10	0.14	0.62
Analyst, failing step only	0.30	0.32	0.07	0.10	0.32	0.23	0.49	0.19	0.10	0.43
Agent model, full trajectory	0.66	0.55	0.22	0.11	0.16	0.58	0.71	0.21	0.24	0.40
Agent model, failing step only	0.28	0.43	0.14	0.10	0.35	0.22	0.67	0.36	0.09	0.39

Table 8: Single-step checks with four judges. Cost is per check as a fraction of a full replay (uncached input plus output tokens), including regenerating the failing step (0.15 of a replay on retail, 0.12 on airline). Sensitivity and false-positive rate are against the paired full-replay outcome of the same patch and trace. Break-even is the cost ratio below which the check decides an edge at lower expected cost than replays (Proposition 4, with the domain’s mean spurious recovery and median repair rate of effective edges, averaged over true and false edges); Ratio is break-even divided by cost, so values below 1 mean replays are more informative per unit cost. The bank judge row uses its own regeneration on the same 360 pairs. Each domain has 360 pairs, 180 with a successful replay; the agent-model full-trajectory judge was run on a random 40 of them because it costs about half a replay per check.

best on retail at $c = 0$ (on airline, CARVE told $c = 0$ edges it, 0.71 against 0.60), replay-each at $c = 0.01$ on retail, the per-patch HarnessFix-style rule from $c = 0.01$ to 0.02 on airline and at 0.02 on retail, and the net-effect method at $c = 0.05$ on retail under both cost types and on airline per applied change. Charging every applied change rather than only wrong ones lowers every policy but changes the winner in only one setting (airline, $c = 0.05$). Told $c = 0$, the policies that use a cost internally accept patches with significant regressions, which is why some of them are negative in the first column.

I Held-Out Ablations

These runs were made on retail, on the same held-out episodes as Table 3 (Tables 13 and 14) or on new clean episodes (Table 15). Differences

are paired and task-clustered, with 90% intervals. Leaving any one of the seven attributed patches out lowers success by 0.00 to 0.03, and the sum of these contributions is not distinguishable from the whole set’s effect, so the runs neither show nor exclude interactions. On clean episodes, applying everything gains +0.04 [−0.03, 0.10] over the unpatched agent. The bundle gate is ahead of it by +0.04 [0.01, 0.08], but three of its five seed sets are identical to the attributed set and one is empty, so the difference is not an effect of verification; on the 72 slots where it ran the identical set the difference is +0.03 [−0.04, 0.09], which indicates the run-to-run noise of these comparisons. Unpatched clean success was 0.82 both before and after the new runs.

Harm model	Retail					Airline				
	μ	Apply	Best	P	VPI	μ	Apply	Best	P	VPI
Hierarchical normal (primary)	−0.00	0.77	1.05	0.63	2.02	−0.01	0.73	0.71	0.46	0.69
Complete pooling ($\tau = 0$)	−0.00	1.12	1.02	0.36	0.87	−0.01	0.73	0.71	0.46	0.69
Spread doubled (2τ)	−0.00	0.27	0.96	0.70	2.51	−0.01	0.73	0.71	0.46	0.69
No pooling	0.04	−1.82	0.20	0.82	4.65	0.04	0.39	0.37	0.43	1.15
Tasks resampled	−0.00	−0.52	0.58	0.65	3.31	−0.00	0.63	0.62	0.45	0.87
No support cap	−0.00	0.82	1.09	0.64	9.48	−0.01	0.74	0.71	0.45	1.06
Not deployable	−0.00	0.64	1.14	0.61	2.15	−0.01	0.86	0.86	0.47	0.56

Table 9: Sensitivity of the posterior to the harm model, $c = 0$, budget 40: mean μ , value of Apply attributed and of the best of the five verifying policies of Table 7, the probability that it beats Apply attributed, and the total value of perfect information over Apply attributed (fractions of the oracle). Airline has $\tau = 0$, so pooling and doubling τ change nothing there.

Regression term	Retail			Airline		
	10	40	80	10	40	80
As used (clip, rescale)	0.70	0.90	1.08	0.63	0.70	0.77
No clip	0.65	0.38	0.52	0.78	0.82	0.81
No rescale	0.70	0.90	1.08	0.63	0.70	0.77
Neither (Eq. 1)	0.63	0.40	0.56	0.79	0.82	0.81

Table 10: Net-effect method with its regression term as used (clipped at zero, scaled by $1/(1 - r_0)$) and without either, posterior value of deployable sets at $c = 0$ over 40 seeds.

Verified set (budget 10)	Gain [s.e.]	$N=300$	$N=1,000$	$N=10,000$	Ever
<i>Retail</i>					
Replay-each	−0.04 [0.04]	0.02	0.09	0.14	0.15
HarnessFix-style, per patch	−0.04 [0.05]	0.14	0.24	0.29	0.30
Uncertainty sampling	−0.02 [0.03]	0.07	0.23	0.32	0.33
Uncertainty + checks	−0.03 [0.03]	0.04	0.15	0.21	0.22
CARVE, replay only	0.00 [0.03]	0.21	0.45	0.55	0.56
CARVE	−0.04 [0.04]	0.05	0.13	0.19	0.20
CARVE, measured check	−0.03 [0.03]	0.03	0.13	0.20	0.22
CARVE, robust check	−0.01 [0.03]	0.10	0.28	0.38	0.39
Net-effect (ours)	−0.04 [0.03]	0.03	0.10	0.15	0.16
HarnessFix-style, bundle	−0.03 [0.01]	0.00	0.02	0.04	0.05
<i>Airline</i>					
HarnessFix-style, per patch	−0.04 [0.05]	0.13	0.23	0.28	0.29
HarnessFix-style, bundle	−0.03 [0.04]	0.10	0.18	0.23	0.23
CARVE	−0.01 [0.05]	0.23	0.39	0.46	0.47
Net-effect (ours)	−0.00 [0.05]	0.28	0.44	0.50	0.51

Table 11: Probability that a verified set has paid back its verification episodes after N future episodes, relative to applying the attributed patches. Gain is the held-out paired difference in success per episode (task-clustered s.e.); each draw adds the change cost saved by applying fewer patches, with $c_a \sim U[0, 0.02]$ of the failure mass per change. “Ever” is the probability that the per-episode gain is positive.

net-effect method of §5 was developed afterwards, on retail, from the diagnosis that CARVE lacked per-category spurious recovery and evidence for regressions.

K ShopDesk Details

ShopDesk has six deterministic tools (orders, products, policies, refunds), answers checked exactly (refund amounts, counts, totals), and five injected faults: a tool description misstating price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that skips the policy lookup, and a tight step limit. Over three seeds, the analyst’s attribution accuracy on injected failures was 0.75, 0.64 and 0.77. ShopDesk was evaluated online, without a bank: each policy ran against the live agent with fault-based ground truth (an edge is real when at least 30% of a category’s failures come

J Development Variants

On the retail bank we tried, before any airline evaluation of them:

- CARVE with two null replays per category before trusting its replays (per-category b);
- CARVE with a regression gate of two runs per accepted patch;
- both combined;
- an evidence rule that requires a positive likelihood ratio from data;
- tempered and flat priors;
- a Bayesian regression gate.

With control replays and the gate combined, CARVE gained 0.68 and 0.60 of the oracle at budgets 10 and 20, above plain CARVE (0.42, 0.48), but fell below it at 40 and 80 (0.47 vs. 0.57 and 0.56). It never beat the per-patch HarnessFix-style rule (0.72, 0.73). Post hoc on airline it was worse than plain CARVE (0.19 vs. 0.31 at budget 10). The other variants were worse everywhere; the evidence rule and the Bayesian gate accepted almost nothing (gain ≤ 0.12). These numbers use the fixed change cost c without regressions and the bank before the regression runs were extended. The

Policy (budget 40)		per wrong change c_w				per applied change c_a			
		0	0.01	0.02	0.05	0	0.01	0.02	0.05
Retail	Apply attributed	0.84	0.50	0.16	-0.85	0.84	0.43	0.01	-1.23
	Replay-each	0.74	0.53	0.33	-0.30	0.74	0.48	0.22	-0.57
	HarnessFix-style, per patch	0.51	0.46	0.42	0.28	0.51	0.42	0.34	0.09
	HarnessFix-style, bundle	0.21	-0.05	-0.31	-1.09	0.21	-0.11	-0.42	-1.38
	Uncertainty sampling	-1.90	-2.46	0.23	0.05	-1.90	-2.57	0.09	-0.17
	Uncertainty + checks	-1.57	-1.95	0.25	0.25	-1.57	-2.06	0.12	0.01
	CARVE, replay only	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE	-0.19	-0.19	0.19	0.28	-0.19	-0.27	0.07	0.14
	CARVE, measured check	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE, robust check	-0.05	-0.70	-0.49	0.01	-0.05	-0.81	-0.66	-0.26
	Net-effect (ours)	-1.02	-0.58	0.41	0.63	-1.02	-0.67	0.32	0.44
Airline	Apply attributed	0.60	0.37	0.15	-0.53	0.60	0.26	-0.08	-1.09
	Replay-each	0.09	-0.02	-0.12	-0.43	0.09	-0.08	-0.25	-0.76
	HarnessFix-style, per patch	0.40	0.39	0.37	0.32	0.40	0.33	0.25	0.02
	HarnessFix-style, bundle	0.53	0.32	0.12	-0.50	0.53	0.22	-0.09	-1.02
	Uncertainty sampling	0.49	0.09	0.25	0.19	0.49	-0.04	0.01	-0.25
	Uncertainty + checks	0.44	0.01	0.29	0.21	0.44	-0.12	0.08	-0.13
	CARVE, replay only	0.48	0.08	-0.02	0.06	0.48	-0.07	-0.26	-0.44
	CARVE	0.71	0.33	0.23	0.10	0.71	0.22	0.08	-0.07
	CARVE, measured check	0.56	0.10	0.07	0.11	0.56	-0.04	-0.17	-0.38
	CARVE, robust check	0.63	0.21	-0.01	0.08	0.63	0.06	-0.26	-0.42
	Net-effect (ours)	0.60	0.10	0.31	0.29	0.60	-0.04	0.14	0.05

Table 12: Value at budget 40 as a fraction of the repair-only oracle, with significant regressions charged, as the change cost varies. Policies that use a cost internally are told the same c (the net-effect method always charges it per applied change, the others per wrong change). At $c = 0$ the uncertainty-based policies accept almost every cell with a positive posterior and pick up patches with significant regressions.

Set (seed 0)	With – without step patch			Without – unpatched
	All	No step fault	Step fault	No step fault
Apply attributed	0.03 [-0.04, 0.10]	-0.03 [-0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
HarnessFix-style, bundle	0.03 [-0.04, 0.10]	-0.03 [-0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
Uncertainty sampling	0.01 [-0.07, 0.08]	-0.02 [-0.09, 0.06]	0.25 [-0.07, 0.57]	0.06 [-0.04, 0.15]
CARVE	0.03 [-0.04, 0.11]	0.00 [-0.07, 0.07]	0.33 [0.07, 0.59]	0.05 [-0.03, 0.13]
Net-effect (ours)	0.05 [-0.02, 0.12]	-0.01 [-0.08, 0.06]	0.58 [0.34, 0.83]	0.02 [-0.07, 0.10]

Table 13: Removing the step-limit patch from each policy’s seed-0 set (the bundle gate’s seed-0 set equals the attributed set). 108 episodes without and 12 with the step-limit fault.

Patch left out of Apply attributed	All	No step fault
CFG.max_steps	0.03 [-0.04, 0.10]	-0.03 [-0.10, 0.04]
POL.cancel	0.03 [-0.04, 0.10]	0.04 [-0.04, 0.12]
POL.exchange	0.03 [-0.04, 0.09]	0.03 [-0.05, 0.11]
POL.generic	0.03 [-0.04, 0.09]	0.01 [-0.05, 0.07]
POL.intro	0.02 [-0.05, 0.08]	0.02 [-0.05, 0.08]
POL.modify	0.03 [-0.05, 0.10]	0.04 [-0.05, 0.12]
POL.return	0.00 [-0.06, 0.06]	-0.01 [-0.08, 0.06]
Whole set vs unpatched	0.12 [0.05, 0.19]	0.06 [-0.01, 0.14]
Whole set – sum of parts	-0.04 [-0.35, 0.27]	-0.03 [-0.38, 0.33]

Table 14: Leave-one-out on the attributed set: success of the whole set minus the set without the named patch.

from that component’s fault). Table 16 reports edge F1 and held-out success. Large harms do occur: a formatting patch that passed its target-category replays made 26% of held-out tasks answer N/A (success 0.89→0.73).

	Old	New	Pooled	– Unpatched	– Apply
Unpatched	0.82	0.82	0.82	–	-0.04 [-0.10, 0.03]
Apply attributed	0.87	0.85	0.85	0.04 [-0.03, 0.10]	
Net-effect (ours)	0.83	0.87	0.86	0.04 [-0.01, 0.09]	0.00 [-0.05, 0.06]
HarnessFix-style, bundle	0.87	0.91	0.90	0.08 [0.03, 0.14]	0.04 [0.01, 0.08]
HarnessFix-style, per patch	0.90	0.85	0.86	0.05 [-0.01, 0.10]	0.01 [-0.06, 0.08]

Table 15: Clean held-out episodes: 38 old (Table 3) and 120 new (40 tasks × 3), where new episode e uses the policy’s seed- $(e \bmod 5)$ set. Differences pool both, paired by episode or slot and clustered by task.

L The CARVE Design Study

Why the simulator was optimistic. Our first design study, CARVE, used a simulator in which it needed half the budget of the strongest baseline (Appendix L). Calibrated to the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator still predicts that CARVE leads uncertainty sampling by 14 points on retail at budget 50; on the bank it trails by 15 points at budget 40 (significant harm, $c_w = 0.02$). The simulator shared the method’s structure: one b for all categories, value spread over many edges, and a fixed cost c that made every wrong change harmful, an assumption that was never measured and on which the value of verification largely depends.

Our first approach treated judge checks as cheap and replays as expensive, and allocated a budget

Method	Edge F1 at budget				Held-out success @60	Single checks
	10	20	30	60		
LLM attribution only	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.98 $\pm$ 0.00	0
Replay-each (do-then-verify)	0.62 $\pm$ 0.03	0.63 $\pm$ 0.03	0.67 $\pm$ 0.05	0.75 $\pm$ 0.03	0.99 $\pm$ 0.00	0
Uncertainty sampling	0.55 $\pm$ 0.04	0.57 $\pm$ 0.04	0.58 $\pm$ 0.05	0.66 $\pm$ 0.04	0.99 $\pm$ 0.00	0
Uncertainty + cheap checks	0.52 $\pm$ 0.07	0.56 $\pm$ 0.07	0.53 $\pm$ 0.09	0.53 $\pm$ 0.08	0.96 $\pm$ 0.03	57
CARVE, full replay only	0.61 $\pm$ 0.07	0.61 $\pm$ 0.06	0.59 $\pm$ 0.04	0.64 $\pm$ 0.03	0.91 $\pm$ 0.09	0
CARVE (default check model)	0.59 $\pm$ 0.06	0.57 $\pm$ 0.05	0.61 $\pm$ 0.05	0.68 $\pm$ 0.03	0.98 $\pm$ 0.01	4
CARVE, calibrated check	0.59 $\pm$ 0.06	0.58 $\pm$ 0.05	0.60 $\pm$ 0.05	0.66 $\pm$ 0.04	0.89 $\pm$ 0.08	0
CARVE, robust check model	0.58 $\pm$ 0.07	0.61 $\pm$ 0.06	0.61 $\pm$ 0.04	0.69 $\pm$ 0.01	0.99 $\pm$ 0.01	0
Unpatched agent					0.59 $\pm$ 0.04	

Table 16: ShopDesk (3 seeds, mean $\pm$ s.e.). Held-out success after applying the patches accepted at budget 60 to 120 held-out episodes.

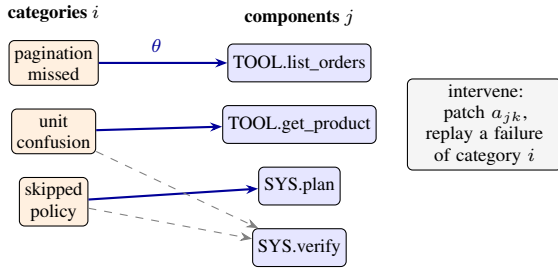


Figure 1: The category-component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

over both by Bayesian experimental design. We keep it here as the multi-fidelity method compared in §7, together with the simulator in which it was developed. The simulator shares the method’s structure. Its results show the method working under its own assumptions, and they are not evidence about real agents.

Calibrated simulation. With the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator predicts that CARVE runs no judge checks on ShopDesk and retail and only a few on airline, which matches the break-even analysis in Table 2. For τ^2 retail it predicts net gains at budget 50 of 0.51 for CARVE and 0.38 for uncertainty sampling; for airline, 0.33 and 0.31; for ShopDesk, 0.81 and 0.79.

L.1 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of fail-

ures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i , applies patch a_{ijk} , and observes a binary outcome at fidelity ϕ . A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk^*} of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (4)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

L.2 CARVE

Observation model For a trajectory labelled i and patch a_{ijk} , the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda) b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (5)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

Attribution-informed prior Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r}) \rho + \hat{r} \min(1, d n_{ij} / n_i), \quad (6)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 6 is a heuristic interpolation, not a calibrated model; with a larger audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

Posterior Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij} and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\begin{aligned} \text{logit } P(e_{ij}=1 \mid \mathcal{D}) &= \text{logit } \pi_{ij} \\ &+ \sum_k \log(\bar{L}_{ijk}^1 / L_{ijk}^0), \end{aligned} \quad (7)$$

where $\bar{L}_{ijk}^1 = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}=1) dP(q)$ is the marginal likelihood of patch k ’s outcomes under a true edge and $L_{ijk}^0 = P(\mathcal{D}_{ijk} \mid e_{ij}=0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

Selecting interventions The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{\text{fp}}, 0), \quad (8)$$

the expected contribution of the cell to Eq. 4 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 4 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (9)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1, 2, 4, 8\}} \frac{\text{KG}_n(a)}{n c_\phi}. \quad (10)$$

Looking ahead several observations matters because a single binary outcome often cannot flip a decision, in which case $\text{KG}_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell’s scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

L.3 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE’s full policy; they explain which quantities govern the method’s behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \parallel p_0)$ and $D_0 = \text{KL}(p_0 \parallel p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald’s approximations (Wald, 1945), are in Appendix L.5.

Proposition 3 (Value of the LLM prior). *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^{ℓ} for a false one). A cell with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho \mathbb{E}[\delta\ell \mid e=1]}{D_1} - \frac{(1 - \rho) \mathbb{E}[\delta\ell \mid e=0]}{D_0}, \quad (11)$$

with $\delta\ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 6, $\ell_{ij} \rightarrow \ell_\rho$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix L.7 compares these predictions with simulated sequential tests.

Proposition 4 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (12)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE’s look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section L.4 tests both predictions.

L.4 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if its own edge to j exists, unlike the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section L.4 relaxes the assumptions the generator shares with the model. Full details are in Appendix L.6.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE’s own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for *Sequential-each*, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the *Replay-each* variants share CARVE’s posterior and decision rule, so comparisons among them isolate the selection rule.

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Replay-each, 3 patches	0.301 $\pm$ 0.025	0.317 $\pm$ 0.026	0.401 $\pm$ 0.026	0.612 $\pm$ 0.026
Sequential-each (adaptive stopping)	0.426 $\pm$ 0.016	0.522 $\pm$ 0.018	0.643 $\pm$ 0.016	0.813 $\pm$ 0.014
Thompson sampling	0.407 $\pm$ 0.020	0.419 $\pm$ 0.020	0.500 $\pm$ 0.017	0.625 $\pm$ 0.014
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
Uncertainty + cheap checks	0.549 $\pm$ 0.018	0.662 $\pm$ 0.016	0.784 $\pm$ 0.015	0.876 $\pm$ 0.009
CARVE (all components)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($m=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 17: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

Metrics. Net gain (Eq. 4) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for the others; all methods see the same worlds, so differences are paired.

Main results Figure 2 and Table 17 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap narrows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks. With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), because it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE’s advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§L.4).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. Replay-each is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small bud-

gets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§L.4).

When does the LLM prior help? Figure 3 varies the analyst’s accuracy r . As Proposition 3 suggests, CARVE improves monotonically with r and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 . The prior is not exactly flat there (the decoy still receives about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

When do cheap checks help? Figure 5 (Appendix L.8) is qualitatively consistent with Proposition 4. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost 0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

Robustness The generator shares several assumptions with the model: check errors depend only on

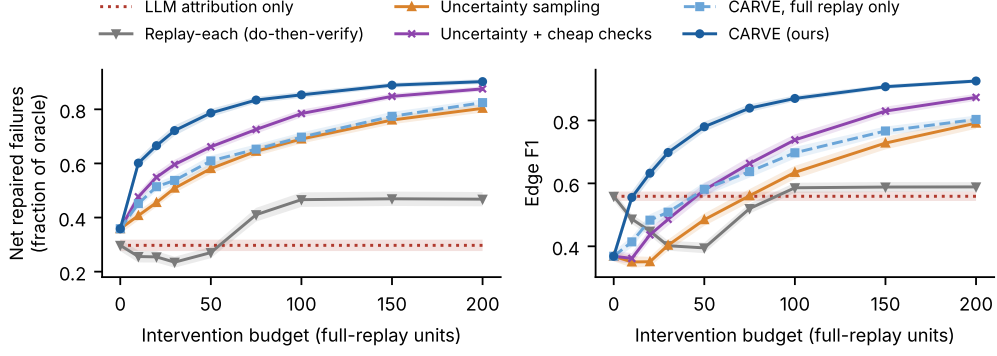


Figure 2: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

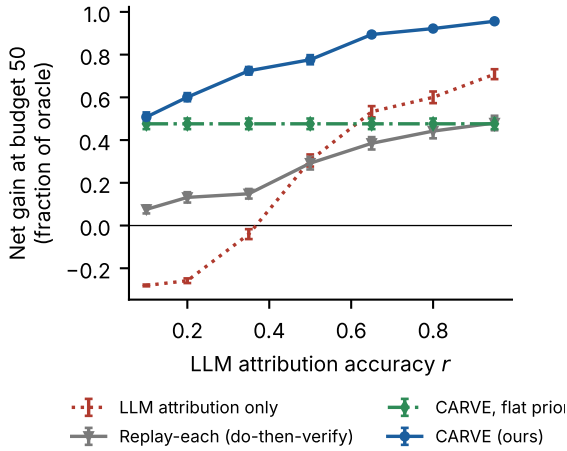


Figure 3: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 ± 0.022	0.776 ± 0.021
Label accuracy $\lambda = 0.5$	0.513 ± 0.023	0.688 ± 0.023
Analyst overconfident ($\hat{r} + 0.3$)	0.752 ± 0.018	0.857 ± 0.014
Analyst underconfident ($\hat{r} - 0.3$)	0.495 ± 0.023	0.653 ± 0.032
Judge fooled by bad patches	0.579 ± 0.022	0.377 ± 0.027
Regression cost $c_{fp} = 0.005$	0.711 ± 0.022	0.798 ± 0.016
Regression cost $c_{fp} = 0.05$	0.643 ± 0.024	0.793 ± 0.019
Regression cost $c_{fp} = 0.1$	0.624 ± 0.023	0.823 ± 0.018

Table 18: Violations of the model’s assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

the full outcome, cells are independent, the analyst’s accuracy is estimated without bias, and the regression cost is small. Table 18 and Appendix L.8 relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst’s accuracy by 0.3, CARVE actually improves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-only falls from 0.39 to -0.13 : the more costly a

wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 21).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b , λ or the patch-quality prior changes the result by at most 5 points (Table 22, Appendix L.8); misstating the check’s sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$,

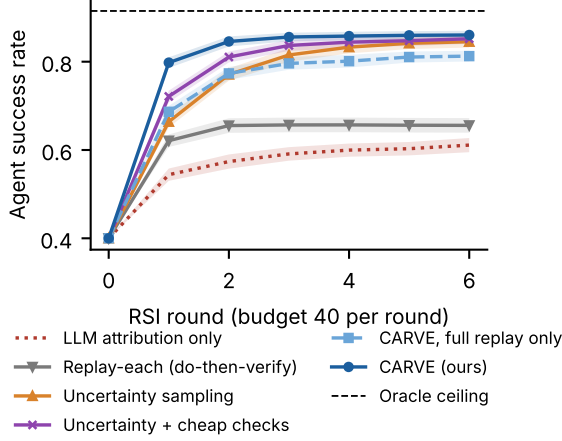


Figure 4: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

$J = 50$; Table 20) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

Inside a self-improvement loop Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 4 shows that CARVE reaches 80% success after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

L.5 Proofs for CARVE’s propositions

Proposition 3. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1||p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald’s approximations give $P_1(L_N \leq -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$

and $P_0(L_N \geq A) \approx \delta e^\ell$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$ no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald’s identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 4. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality gives $D_e^S \leq D_e$, so the critical ratio is at most one.

L.6 Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

L.7 Checking Proposition 1

Table 19 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
+0	1	4.05	4.76	0.012
+0	0	6.75	7.28	0.009
+1	1	3.01	4.27	0.006
+1	0	8.48	8.58	0.015
+2	1	1.97	2.97	0.001
+2	0	10.22	10.23	0.068

Table 19: Proposition 3: predicted vs. simulated observations to a decision.

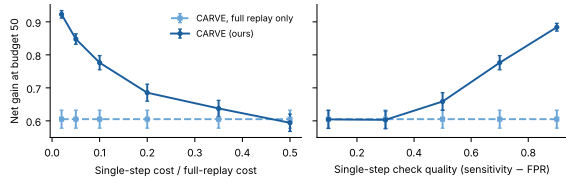


Figure 5: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

L.8 Additional Results

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
Uncertainty + cheap checks	0.603 $\pm$ 0.019	0.746 $\pm$ 0.018	0.873 $\pm$ 0.013
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 20: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028
Replay-each	0.202 $\pm$ 0.027	0.207 $\pm$ 0.023	0.348 $\pm$ 0.033	0.362 $\pm$ 0.036
Thompson sampling	0.325 $\pm$ 0.031	0.343 $\pm$ 0.032	0.398 $\pm$ 0.032	0.484 $\pm$ 0.028
Uncertainty sampling	0.345 $\pm$ 0.036	0.465 $\pm$ 0.035	0.544 $\pm$ 0.035	0.662 $\pm$ 0.029
Uncertainty + cheap checks	0.423 $\pm$ 0.037	0.558 $\pm$ 0.035	0.615 $\pm$ 0.034	0.685 $\pm$ 0.030
CARVE (ours)	0.573 $\pm$ 0.029	0.661 $\pm$ 0.029	0.701 $\pm$ 0.027	0.745 $\pm$ 0.026

Table 21: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 $\pm$ 0.021
Spurious rate b : 0.01 (0.05)	0.811 $\pm$ 0.012
Spurious rate b : 0.15 (0.05)	0.826 $\pm$ 0.016
Label accuracy λ : 1.0 (0.8)	0.768 $\pm$ 0.022
Label accuracy λ : 0.6 (0.8)	0.794 $\pm$ 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 $\pm$ 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 $\pm$ 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 $\pm$ 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 $\pm$ 0.020

Table 22: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several “wrong” values do slightly better than the generator’s, a reminder that the model is misspecified in other ways too.